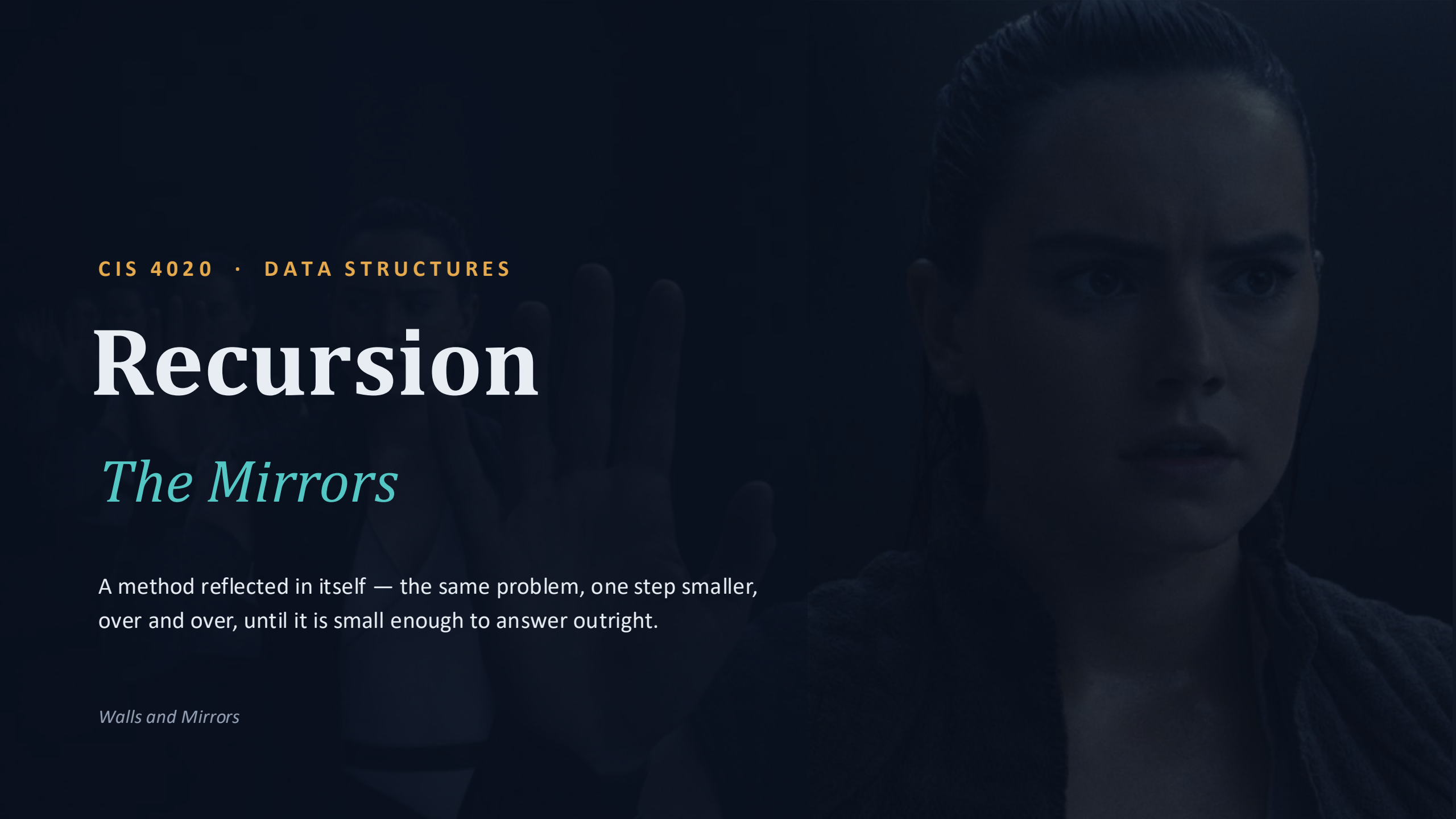
CIS 4020 · DATA STRUCTURES

Recursion

The Mirrors

A method reflected in itself — the same problem, one step smaller, over and over, until it is small enough to answer outright.

Walls and Mirrors



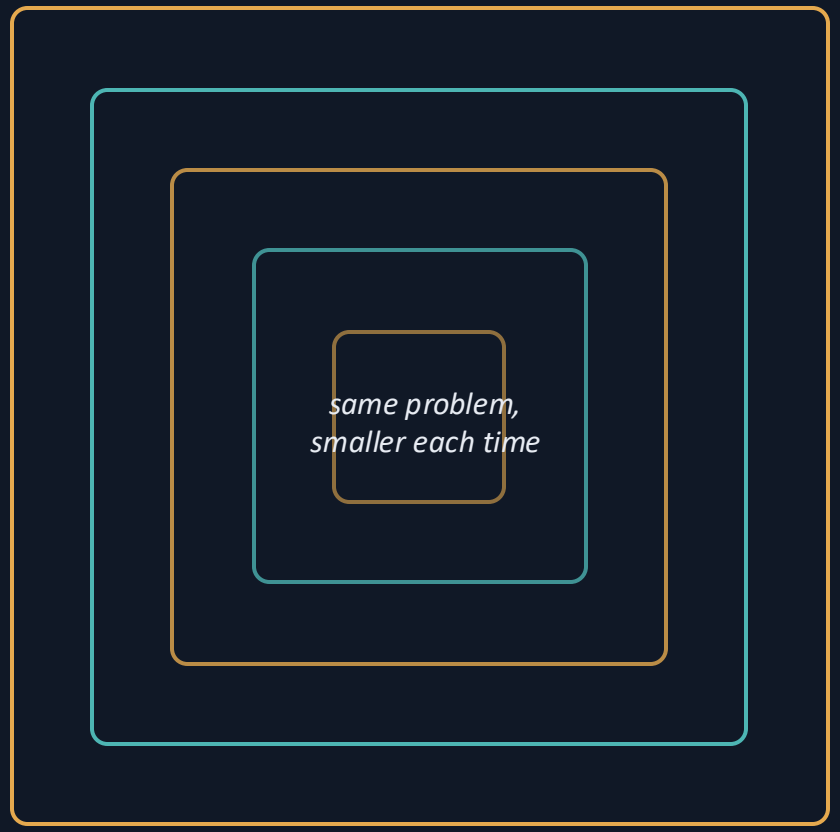
THE IDEA · THE DIVIDE AND CONQUER PARADIGM

What is recursion?

A recursive method solves a problem by **calling itself** on a smaller version of the same problem.

1. **Divide** Break the problem into one or more smaller problems of the same shape.
2. **Conquer** Solve the smaller problem — usually by recursion again.
3. **Combine** Build this answer from the smaller answers on the way back.

Not the same as a loop. *Some problems that are awkward to write with iteration fall out in a few lines when you let the method lean on itself.*



a mirror facing a mirror

ANATOMY

Every recursive solution has three parts

Base case

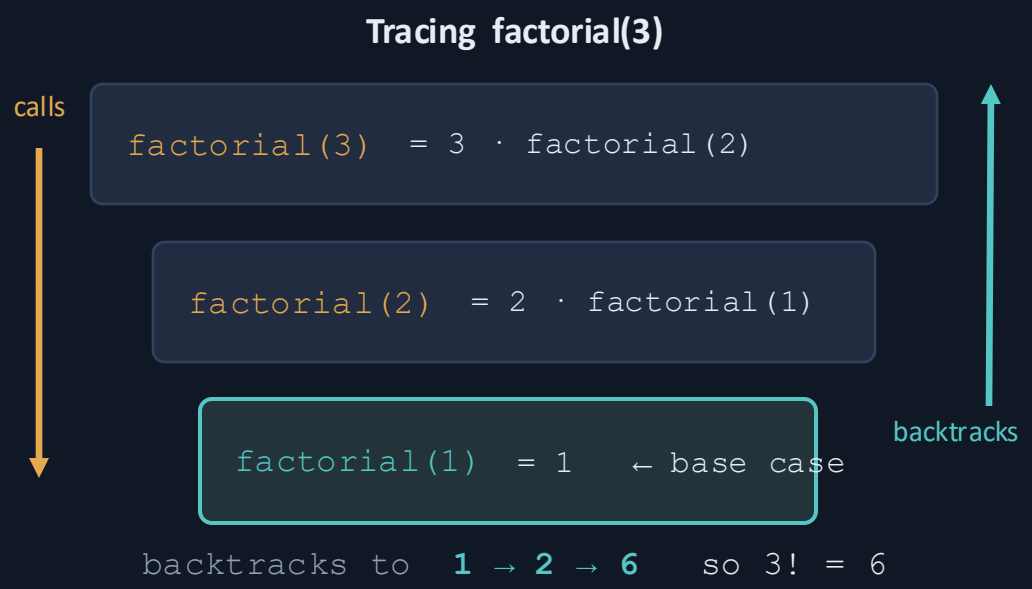
A case small enough to answer directly — no further calls. Every recursion must reach one.

Recursive step

Solve a smaller instance of the same problem, moving closer to the base case.

Backtracking

Each call returns to the one that called it; answers combine as the calls backtrack.



IN CODE

The same shape in both languages

Base case first, then a recursive call on a **smaller** argument. Read both — they are the same method.

Java

```
// Java   (CIS)
long factorial(int n) {
    if (n <= 1)           // base case
        return 1;
    return n * factorial(n - 1);   // smaller
}
```

C / C++

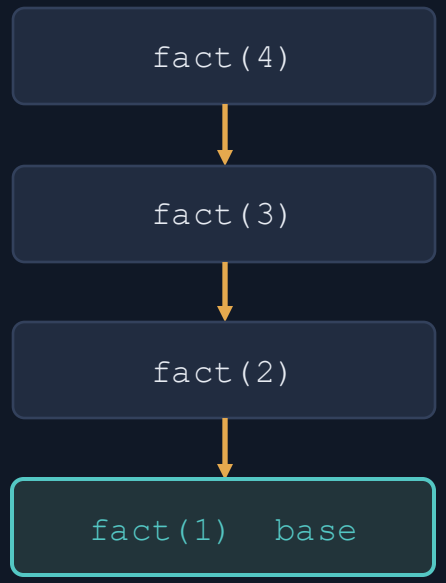
```
// C++   (CS)
long factorial(int n) {
    if (n <= 1)           // base case
        return 1;
    return n * factorial(n - 1);   // smaller
}
```

Read it as a promise: "if you can give me the answer for n − 1, I can give you the answer for n." The base case is where the promises finally get paid.

TWO SHAPES

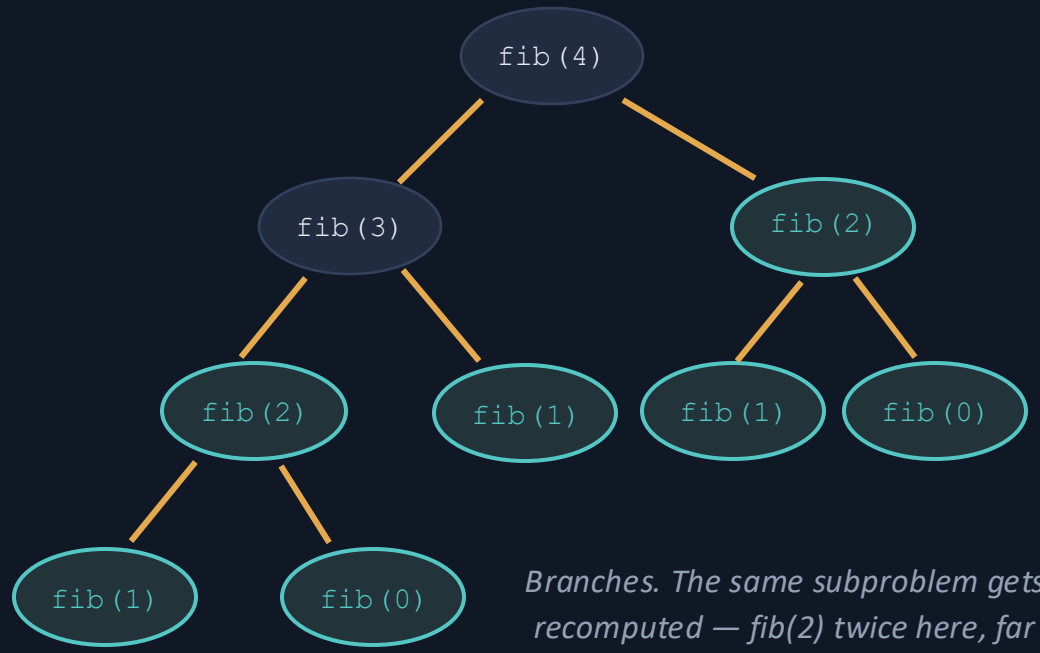
Linear vs. tree recursion

Linear — one call each step



A straight chain. Depth n, work grows with n.

Tree — two or more calls each step



Branches. The same subproblem gets recomputed — fib(2) twice here, far more as n grows.

JUDGMENT

When recursion earns its keep

Good fit

- The problem is defined in terms of itself (trees, nested structures).
- A smaller version is obviously easier and clearly reachable.
- The recursive solution is far simpler to read than the loop.

Costs to respect

Memory — Every call sits on the stack until it returns. Too deep → stack overflow.

Repeated work — Tree recursion can recompute the same subproblem many times (see Fibonacci).

Overhead — Each call costs a little setup. A plain loop is often cheaper when both read equally well.

"When all you have is a hammer…" Recursion is a tool, not a default. If a loop is just as clear and cheaper, use the loop.

WHERE WE GO NEXT

You'll see recursion again, and again, and again.

Next we trace these by hand and code them together in Java and C++. *Study the "kth smallest item in an array" section as a good example.*